

Iteration Methods in HygroThermFEM

Lawrence Berkeley National Laboratory / Oak Ridge National Laboratory

1 Problem Overview

HygroThermFEM solves two coupled partial differential equations over a 2D finite element domain:

- **Thermal equation:** governs temperature distribution, with material properties (thermal conductivity) that may depend on moisture content.
- **Moisture equation:** governs relative humidity distribution, with transport coefficients that depend on temperature.

Because material properties depend on the solution itself, both equations are in general **non-linear**. The coupling between the two domains adds a further layer of nonlinearity: temperature affects moisture transport, and moisture affects thermal conductivity.

The solver addresses this through a hierarchy of three levels:

Level	Responsibility
Outer: <code>MultiDomain::transient</code>	Adaptive time-accumulation with cross-coupling
Middle: <code>IDomain::transient</code>	Adaptive timestep probing
Inner: <code>IDomain::transientTimestep</code>	Newton–Raphson iteration

Table 1: Solver hierarchy

2 Newton–Raphson Iteration (Inner Level)

Each individual domain (thermal or moisture) solves its nonlinear system using a **modified Newton–Raphson (NR) method with relaxation**.

2.1 Algorithm

Given the discretized FEM system $\mathbf{A} \cdot \mathbf{U} = \mathbf{B}$, where both $\mathbf{A}$ (stiffness + mass + boundary matrices) and $\mathbf{B}$ (load vector) depend on the current solution $\mathbf{U}$:

Input: $\mathbf{U}_0$ = initial guess, ω = relaxation parameter, tol = convergence tolerance

1. Assemble $\mathbf{A}(\mathbf{U}_0)$, $\mathbf{B}(\mathbf{U}_0)$
2. For $k = 0, 1, 2, \dots$
 1. Compute residual: $\mathbf{r} = \mathbf{B} - \mathbf{A} \cdot \mathbf{U}_k$
 2. Solve for correction: $\mathbf{A} \cdot \Delta \mathbf{U} = \mathbf{r}$
 3. Limit increment: clamp $\Delta \mathbf{U}$ per DOF (see Section 4.1)
 4. Apply relaxed update: $\mathbf{U}_{k+1} = \mathbf{U}_k + \omega \cdot \Delta \mathbf{U}$
 5. Post-process: enforce physical bounds on $\mathbf{U}_{k+1}$
 6. Update node properties with $\mathbf{U}_{k+1}$
 7. Reassemble: $\mathbf{A}(\mathbf{U}_{k+1})$, $\mathbf{B}(\mathbf{U}_{k+1})$
 8. Check convergence: metric = $\left| \frac{\|\mathbf{U}_{k+1}\| - \|\mathbf{U}_k\|}{\|\mathbf{U}_{k+1}\|} \right|$
 9. If metric $\leq$ tol: converged, exit
10. If all DOFs clamped: converged at physical bounds, exit (see Section 4.1)
11. If oscillation detected: apply midpoint averaging (see Section 4.2), exit
12. If $k > \text{MaxIterations}$: not converged, exit

The convergence criterion is based on the **relative change in the L^2 norm** of the solution between consecutive iterations.

2.2 Linear Problems

When both boundary conditions and element properties are linear (no state-dependent materials), the system $\mathbf{A} \cdot \mathbf{U} = \mathbf{B}$ is solved in a single direct step without iteration.

3 Adaptive Time-Accumulation with Cross-Coupling (Outer Level)

The coupled hygrothermal problem is solved using a **partitioned scheme with adaptive time-stepping at the multi-domain level**. Rather than assembling a monolithic system, each domain is solved independently and then cross-domain data is exchanged after every successful sub-step.

3.1 Design Rationale

Earlier versions of the solver used a staggered coupling loop where each domain internally subdivided the timestep as needed, grinding through potentially thousands of tiny sub-steps with the cross-domain field (temperature or humidity) frozen at its initial value. For extreme conditions (high humidity at elevated temperatures), this led to:

- **Excessive computation:** the moisture domain might require 40,000+ internal sub-steps to cover a single 3600s timestep
- **Reduced accuracy:** moisture transport coefficients depend on temperature, but that temperature was frozen during all internal sub-steps
- **Poor coupling convergence:** the staggered outer loop had to repeat the full internal solve to capture cross-domain effects

The current approach moves the time-accumulation loop out of the individual domains and into `MultiDomain::transient`. Each domain now returns after its **first successful solve** (at whatever sub-timestep it could converge), and cross-coupling data is exchanged after every sub-step. This ensures both domains always see each other’s latest solution.

3.2 Transient Coupling Algorithm

Input: T_0 = initial temperature, H_0 = initial humidity, Δt = target timestep

1. $t_{\text{total}} = 0$
2. While $t_{\text{total}} < \Delta t$:
 1. $t_{\text{remain}} = \Delta t - t_{\text{total}}$, $\Delta t_{\text{eff}} = t_{\text{remain}}$
 2. Solve moisture domain at Δt_{eff} :
 - Domain probes Δt_{eff} , subdivides if NR fails, returns first successful solution with actual Δt used
 - $\Delta t_{\text{eff}} = \min(\Delta t_{\text{eff}}, \Delta t_{\text{moisture}})$
 3. Solve thermal domain at Δt_{eff} :
 - If thermal needs even smaller Δt : set $\Delta t_{\text{eff}} = \Delta t_{\text{thermal}}$, re-solve moisture
 4. Accept both solutions, advance time:
 - $T_{\text{current}} = T_{\text{new}}$, $H_{\text{current}} = H_{\text{new}}$
 - Update nodes with both fields (cross-coupling exchange)
 - $t_{\text{total}} += \Delta t_{\text{eff}}$
3. Final update: advance “previous timestep” values in nodes

3.3 Key Properties

Adaptive synchronization. Both domains always advance by the same sub-timestep. When the moisture domain (typically the stiffer one) needs a smaller Δt , the thermal domain is solved at the same reduced Δt . If the thermal domain requires an even smaller step (rare in practice), moisture is re-solved at that step.

Frequent coupling exchange. Cross-domain data is refreshed after every successful sub-step, not after a full Δt traversal. This dramatically improves convergence for strongly coupled problems: instead of solving moisture with a frozen temperature field, the temperature is updated at every sub-step.

Performance improvement. For the extreme test case (80 °C, RH=0.9999), the new approach reduced solve time from 15.5 s (40,000 level-2 sub-steps with frozen coupling) to 0.3 s (a few dozen sub-steps with live coupling) — approximately 50× faster.

3.4 Steady-State Coupling

The steady-state coupling follows a simpler pattern: alternately solve each domain and exchange data, repeating until both converge. No time-accumulation is needed.

4 Convergence Stabilization Techniques

Near-saturation humidity conditions (RH close to 1.0) present particular challenges for convergence. Several stabilization techniques are employed.

4.1 Per-DOF Increment Limiting

Problem. The Newton–Raphson correction ΔU can project the humidity solution outside the physical range $[0, 1]$. Post-process clamping introduces an inconsistency: the system matrices were assembled assuming an unconstrained state, but the actual solution is clamped. This causes the solver to repeatedly overshoot and clamp, creating artificial oscillations.

Solution. Before applying the correction, each DOF’s increment is scaled so the projected value $U_i + \omega \cdot \Delta U_i$ remains within $[0, 1]$:

$$\text{For each DOF } i : \quad \begin{cases} \Delta U_i = \frac{1-U_i}{\omega} & \text{if } U_i + \Delta U_i \cdot \omega > 1 \\ \Delta U_i = -\frac{U_i}{\omega} & \text{if } U_i + \Delta U_i \cdot \omega < 0 \end{cases}$$

Early convergence via saturation detection. `limitIncrement` returns a flag indicating whether **all** DOF corrections were clamped to effectively zero ($|\Delta U_i| < 10^{-12}$). When true, the solution is pinned at physical bounds and no correction can improve it further. The NR loop treats this as converged immediately.

4.2 Oscillation Detection and Midpoint Averaging

Problem. Piecewise-linear material data (e.g., sorption isotherms) can have steep kinks where the derivative changes abruptly. When the NR iteration crosses such a kink, the Jacobian changes discontinuously, causing the solution to alternate between two states on consecutive iterations (a “2-cycle”).

Detection. After a minimum warmup period (4 iterations), the solver compares the convergence metric between consecutive iterations. If the relative change is less than 1%:

$$|\text{metric}_k - \text{metric}_{k-1}| \frac{1}{\text{metric}_{k-1}} < 0.01$$

the solver concludes that a 2-cycle is present.

Resolution. The solution is set to the midpoint of the last two iterates:

$$U_{\text{final}} = \frac{1}{2}(U_k + U_{k-1})$$

This places the solution at the center of the oscillation band, which lies at (or very near) the material property kink.

4.3 Smooth Vapor Transfer Coefficient

Problem. The water vapor transfer coefficient β originally had a hard step discontinuity at $\text{RH} = 1.0$. During NR iteration, small overshoots past $\text{RH} = 1.0$ would zero out the transfer coefficient, then on the next iteration it would jump back to full value, compounding oscillations.

Solution. Replace the step function with a smooth linear ramp:

$$\text{smoothFactor} = \text{clamp}\left(\frac{1 + w - \text{RH}}{w}, 0, 1\right), \quad w = 0.01$$

$$\beta = \beta_{\text{value}} \cdot \text{smoothFactor}$$

This produces: full transfer below $\text{RH} = 1.0$, a linear taper over $[1.0, 1.01]$, and zero transfer above $\text{RH} = 1.01$.

5 Adaptive Timestep Probing (Middle Level)

When the NR iteration fails to converge at a requested timestep, the domain **subdivides** and tries progressively smaller timesteps until one succeeds, then **returns immediately** with the first successful solution and the actual Δt used:

Input: Δt = requested timestep, maxLevels = 3, numSub = 10

1. $\Delta t_{\text{current}} = \Delta t$, level = 0
2. Loop:
 1. Attempt NR solve for $\Delta t_{\text{current}}$
 2. If converged: return (solution, $\Delta t_{\text{current}}$) $\leftarrow$ actual Δt may be $<$ requested
 3. If not converged: $\Delta t_{\text{current}} = \Delta \frac{t_{\text{current}}}{\text{numSub}}$, level += 1
 4. If level $>$ maxLevels: throw error

The domain **does not accumulate time internally**. It finds the largest Δt at which the NR solver converges and returns that single-step result. The caller (`MultiDomain::transient`) is responsible for accumulating sub-steps until the target time is reached, exchanging cross-domain coupling data between each sub-step (see Section 3).

With default settings (3 levels, 10 subdivisions per level), the solver can probe timesteps as small as $\Delta t/1000$ before reporting failure.

6 Diagnostic Output

Both `IDomain` and `MultiDomain` support an optional diagnostic stream via `setDiagnosticStream`. When enabled, the solver writes CSV-formatted data for every NR iteration and sub-step.

NR iteration columns: `subtimestep_dt`, `nr_iter`, `metric`, `converge_tol`, `correction_norm`, `damping_factor`, `all_clamped`, `converged`, `reason`, `u0`, `u1`, ...

Comment lines (`#`) mark sub-step boundaries, domain switches, and time-accumulation progress.

```
std::ofstream diagFile("solver_diagnostics.csv");
multiDomain.setDiagnosticStream(&diagFile);
```

Load in Python: `pd.read_csv("solver_diagnostics.csv", comment="#")`

7 Summary of Parameters

Parameter	Default	Description
Relaxation parameter	1.0	NR under-relaxation factor (ω)
Error tolerance	10^{-5}	Convergence threshold for relative norm change
Max iterations	50	Maximum NR iterations per timestep
Max division levels	3	Maximum timestep probing depth
Subdivisions per level	10	Factor by which Δt is reduced per probing level
Oscillation check threshold	0.01	Relative metric change below which 2-cycle is detected
Min iterations for oscillation check	4	Warmup before oscillation detection activates
Clamp tolerance	10^{-12}	Absolute increment below which a DOF is considered clamped
Vapor transfer transition width	0.01	RH range over which β tapers to zero

Table 2: Solver configuration parameters